高効用密集区間：頻度・効用同時時系列マイニング

Anonymous Author(s) 査読中

***Abstract*—**頻出アイテムセットマイニングと高効用アイテムセットマイニングは、トランザクションデータ分析の相補的な側面に対処する。前者は共起頻度の高いアイテムを特定し、後者は経済的に重要なアイテム組み合わせを捕捉する。しかし、いずれのフレームワークも時間的な集中—特定のアイテムセットが高頻度かつ高効用を同時に示す時間区間—を検出するようには設計されていない。本研究では、スライディングウィンドウに基づく密集区間検出に効用次元を拡張した効用密集区間（**Utility-Dense Interval, UDI**）フレームワークを提案する。ウィンドウ効用（**WU**）およびウィンドウトランザクション加重利用度（**WTWU**）を定義し、**WTWU**がウィンドウレベルの効用枝刈りのための反単調上界を与えることを証明し、頻度閾値と効用閾値を同時に適用する**Apriori**型アルゴリズムを提示する。合成データおよび半合成データに対する実験により、**(i)** 本アルゴリズムが注入された高効用密集パターンを複数シードにわたり**100%**の精度で復元すること、**(ii) TWU**ベースの枝刈りが候補評価を削減すること、**(iii)** 実行時間がトランザクション数に対してほぼ線形にスケールすることを実証する。本フレームワークは時系列パターンマイニングと効用考慮型分析の交差領域における空白を埋めるものである。

***Index Terms*—**高効用アイテムセットマイニング**,** 密集区間**,** 時系列マイニング**,** スライディングウィンドウ**,** トランザクション加重利用度

I. はじめに

トランザクションデータベースは、小売分析、Webアクセスマイニング、推薦システムの中核をなす。このようなデータから実用的な知見を抽出するために、主に二つの研究系譜が発展してきた。すなわち、頻出アイテムセットマイニング（FIM） [1]と高効用アイテムセットマイニング（HUIM） [2], [3]である。

FIMは、最小支持度以上のトランザクションで共起するアイテムセットを特定するが、すべてのアイテムとトランザクションを等しく扱う。計算論的にはよく理解されているものの、FIMでは低利益率で頻繁に購入される商品と高利益率でたまに購入される商品を区別できない。HUIMはこの限界に対処するため、各アイテムに外部効用（例：単位あたり利益）を、各アイテム出現に内部効用（例：購入数量）を関連付け、総効用が閾値を超えるアイテムセットをマイニングする [4], [5]。

しかし、FIMもHUIMも本質的に非時間的である。すなわち、データベース全体に対して単一のグローバルな結果を生成する。実際には、購買パターンは非常に非定常的である。販促キャンペーン、季節効果、サプライチェーンの混乱が、特定のアイテムセットが一時的に密集化または高効用化する時間区間を生み出す。このような時間的な集中を検出するには、頻度・効用・時間的局所性を同時に考慮するフレームワークが必要である。

密集区間検出。近年のスライディングウィンドウ密集区間マイニングに関する研究は、アイテムセットの出現頻度がすべてのウィンドウ位置で閾値を超える連続時間区間を特定する。このアプローチは共同購買活動の「バースト」を効果的に捕捉する。しかし、頻度のみに基づいており、区間を経済的重要性によってランク付けやフィルタリングすることはできない。

本研究の貢献。我々は、スライディングウィンドウ密集区間検出と効用考慮型マイニングを組み合わせた効用密集区間（UDI）フレームワークを提案する。具体的に、以下の貢献を行う：

1) 効用密集区間問題を形式的に定義し、ウィンドウ効用（WU）およびウィンドウトランザクション加重利用度（WTWU）を導入する（III節）。
2) WTWUがウィンドウ効用に対する反単調上界であることを証明し、候補空間の効果的な枝刈りを可能にする（IV節）。
3) 頻度閾値と効用閾値を同時に適用するApriori型レベルワイズアルゴリズムを提示し、$O(1)$ウィンドウ効用クエリのための累積和最適化を導入する（IV節）。
4) 合成データおよび半合成データに対してアルゴリズムを評価し、高いパターン復元率、効果的なTWU枝刈り、およびほぼ線形のスケーラビリティを実証する（V節）。

II. 関連研究

A. 高効用アイテムセットマイニング

高効用アイテムセットマイニング問題はYaoら [2]により形式化され、内部効用（数量）と外部効用（単位利益）を組み合わせた効用尺度が導入された。支持度とは異なり、効用は反単調ではない。すなわち、高効用アイテムセットの部分集合がその上位集合よりも高い効用を持つ場合がある。このため、新たな枝刈り戦略が必要となった。

トランザクション加重利用度（TWU）モデル [4]は、上界を定義することで反単調性の一形態を回復した。アイテムセットXのTWUは、Xを含むすべてのトランザクションのトランザクション効用の総和である。TWU$(X) \geq u(X)$かつTWUが反単調であるため、見込みのないアイテムセットを早期に枝刈りできる。

HUI-Miner [4]は効用リスト構造を導入し、候補生成なしの一段階マイニングを可能にした。EFIM [5]はトランザクション併合とサブツリー効用上界によりさらに効率を改善した。Tsengら [6]は簡潔表現を研究した。Fournier-Vigerら [3]が包括的なサーベイを提供している。

B. 時間的・周期的効用マイニング

Lanら [7]は棚卸期間をHUIMに組み込み、棚卸期間中に高効用であるアイテムセットをマイニングした。Damら [8]は周期性制約付きの系列パターンに効用マイニングを拡張した。Ahmedら [9]は木構造による増分データベースに対処した。これらの研究は時間的側面を考慮しているが、スライディングウィンドウによる連続的な密集区間の特定は行っていない。

C. 密集区間マイニング

スライディングウィンドウ密集区間検出は、固定サイズのウィンドウ内でアイテムセットの支持度がすべての位置で閾値を超える時間区間を特定する。Apriori-windowフレームワークは、これをレベルワイズ候補生成と組み合わせ、頻度ベースの密集区間の反単調性を枝刈りに活用する。しかし、既存の密集区間手法は純粋に頻度ベースである。

D. 本研究の位置づけ

本研究は、効用考慮型マイニングと時間的密集区間検出の間の空白を橋渡しする。我々の知る限り、効用密集区間—スライディングウィンドウの頻度条件と効用条件を同時に満たす時間区間—を定義しマイニングする先行研究は存在しない。

III. 準備と問題定義

A. 効用付きトランザクションデータベース

定義 1 (効用トランザクションデータベース)**.** 効用トランザクションデータベース $\mathcal{D} = \{T_0, T_1, \ldots, T_{n-1}\}$ はトランザクションの順序付き列である。各トランザクションT_tは時間位置tに対応し、アイテム集合$I(T_t) \subseteq \mathcal{I}$を含む。ここで$\mathcal{I}$はアイテムの全体集合である。各アイテム$i \in I(T_t)$は内部効用（数量）$q(i, T_t) \in \mathbb{Z}_{>0}$を持つ。各アイテム$i \in \mathcal{I}$は外部効用（単位利益）$p(i) \in \mathbb{R}_{>0}$を持つ。

定義 2 (アイテムおよびアイテムセットの効用)**.** トランザクションT_tにおけるアイテムiの効用は$u(i, T_t) = q(i, T_t) \cdot p(i)$である。トランザクション$T_t$におけるアイテムセット$X \subseteq \mathcal{I}$の効用は：

$$u(X, T_t) = \begin{cases} \sum_{i \in X} u(i, T_t) & \textit{if } X \subseteq I(T_t) \\ 0 & \textit{otherwise} \end{cases} \tag{1}$$

T_tのトランザクション効用は$TU(T_t) = \sum_{i \in I(T_t)} u(i, T_t)$である。

B. スライディングウィンドウと密集区間

定義 3 (スライディングウィンドウ)**.** ウィンドウサイズ$W \in \mathbb{Z}_{>0}$に対して、位置lにおけるウィンドウはサブデータベース$\mathcal{D}[l, l{+}W] = \{T_t \mid l \leq t \leq l + W\}$である。

定義 4 (ウィンドウ支持度)**.** 位置lにおけるアイテムセットXのウィンドウ支持度は：

$$\sigma_W(X, l) = |\{T_t \in \mathcal{D}[l, l{+}W] \mid X \subseteq I(T_t)\}| \tag{2}$$

定義 5 (密集区間)**.** アイテムセットXは、ウィンドウサイズWおよび頻度閾値θ_fに対して、すべての$l \in [s, e]$で$\sigma_W(X, l) \geq \theta_f$を満たすとき、密集区間$[s, e]$を持つ。 密集区間$[s, e]$が極大であるとは、同じ条件を満たす真の拡張$[s', e'] \supsetneq [s, e]$が存在しないことをいう。

C. ウィンドウ効用と*TWU*

定義 6 (ウィンドウ効用)**.** 位置lにおけるアイテムセットXのウィンドウ効用は：

$$WU(X, l) = \sum_{t=l}^{l+W} u(X, T_t) \tag{3}$$

定義 7 (ウィンドウトランザクション加重利用度)**.** 位置lにおけるアイテムセットXのウィンドウTWUは：

$$WTWU(X, l) = \sum_{\substack{t=l \\ X \subseteq I(T_t)}}^{l+W} TU(T_t) \tag{4}$$

D. 問題定義

定義 8 (効用密集区間)**.** アイテムセットXは、パラメータ(W, θ_f, θ_u)に対して、すべての$l \in [s, e]$で以下を満たすとき、効用密集区間$[s, e]$を持つ：

1) $\sigma_W(X, l) \geq \theta_f$ （頻度条件）
2) $WU(X, l) \geq \theta_u$ （効用条件）

問題定式化。 効用トランザクションデータベース$\mathcal{D}$、ウィンドウサイズW、頻度閾値θ_f、効用閾値θ_u、最大アイテムセット長$k_{\max}$が与えられたとき、$[s, e]$がXの極大効用密集区間であり$|X| \leq k_{\max}$を満たすすべてのペア$(X, [s, e])$を求めよ。

IV. 提案手法

A. 理論的基礎

まず、効率的な枝刈りを可能にする性質を確立する。

性質 1 (TWU上界)**.** 任意のアイテムセットXおよびウィンドウ位置lに対して：

$$WU(X, l) \leq WTWU(X, l) \tag{5}$$

Proof. $l \leq t \leq l + W$かつ$X \subseteq I(T_t)$を満たす各トランザクションT_tに対して：

$$u(X, T_t) = \sum_{i \in X} u(i, T_t) \leq \sum_{i \in I(T_t)} u(i, T_t) = \mathrm{TU}(T_t)$$

該当するすべてのトランザクションについて総和をとると$\mathrm{WU}(X, l) \leq \mathrm{WTWU}(X, l)$を得る。 □

補題 1 (WTWUの反単調性)**.** 任意のアイテムセット$X \subseteq Y$およびウィンドウ位置lに対して：

$$WTWU(Y, l) \leq WTWU(X, l) \tag{6}$$

Proof. $Y \supseteq X$であるから、Yを含むすべてのトランザクションはXも含む。したがって$\{T_t \mid Y \subseteq I(T_t)\} \subseteq \{T_t \mid X \subseteq I(T_t)\}$であり、部分集合上の$\mathrm{TU}(T_t)$の総和は上位集合上の総和以下となる。 □

定理 1 (UDI枝刈り)**.** $\max_l WTWU(X,l) < \theta_u$ならば、$X$は効用密集区間を持たず、$X$の任意の上位集合$Y \supseteq X$もまた効用密集区間を持たない。

Proof. 性質 1より、すべてのlに対して$\text{WU}(X,l) \leq \text{WTWU}(X,l) < \theta_u$であるから、$X$はすべての位置で効用条件に違反する。補題 1より、すべてのlに対して$\text{WTWU}(Y,l) \leq \text{WTWU}(X,l) < \theta_u$であるから、$Y$もまた枝刈りできる。 □

補題 2 (頻度に関する密集区間の反単調性)**.** アイテムセットXが頻度密集区間を持たないならば、Xの上位集合$Y \supseteq X$も頻度密集区間を持たない。

Proof. $Y \supseteq X$より、Yを含むトランザクションはXを含むトランザクションの部分集合であるから、すべてのlに対して$\sigma_W(Y,l) \leq \sigma_W(X,l)$が成り立つ。 □

系 1 (同時枝刈り)**.** アイテムセットXは、*(a)* 頻度密集区間を持たない、または *(b)* すべてのウィンドウ位置にわたる*WTWU*の最大値がθ_u未満である、のいずれかの場合に枝刈りできる。

B. 累積和による最適化

各位置でウィンドウ効用をゼロから再計算することを避けるため、累積和を事前計算する。

定義 9 (効用累積和)**.** アイテムセットXに対して次を定義する：

$$P_u(X,t) = \sum_{j=0}^{t-1} u(X,T_j) \tag{7}$$

このとき$WU(X,l) = P_u(X,l{+}W{+}1) - P_u(X,l)$を$O(1)$時間で計算できる。

同様に、TWU累積和により$O(1)$でウィンドウTWUクエリが可能となる：

$$P_{\text{TWU}}(X,t) = \sum_{\substack{j=0 \\ X \subseteq I(T_j)}}^{t-1} \text{TU}(T_j) \tag{8}$$

C. アルゴリズム：*UDI-Miner*

Algorithm 1に本レベルワイズマイニング手順を示す。

D. 計算量解析

定理 2 (時間計算量)**.** $n = |\mathcal{D}|$、$m = |\mathcal{I}|$、Kを評価された候補数とする。*UDI-Miner*の時間計算量は$O(K \cdot n + m \cdot n)$である。

Proof. アイテム-トランザクション対応表の構築には$O(n \cdot \bar{d})$を要する。ここで$\bar{d}$は平均トランザクションサイズである。各候補に対して、累積和の構築は$O(n)$、ウィンドウ走査は$O(n)$である。密集区間の計算は$O(c \log c)$で、cは出現回数である。レベルkでの候補生成は$O(|L_{k-1}|^2)$で抑えられる。支配的なコストは、すべての候補にわたる累積和構築と効用検査の$O(K \cdot n)$である。 □

累積和最適化により、各ウィンドウの効用クエリが$O(W)$から$O(1)$に削減され、素朴なウィンドウ走査と比較してW倍の高速化が達成される。

Algorithm 1 UDI-Miner：効用密集区間マイニング

Require: データベース$\mathcal{D}$、外部効用p、ウィンドウサイズW、頻度閾値θ_f、効用閾値θ_u、最大長$k_{\max}$
Ensure: 効用密集区間ペア$(X,[s,e])$の集合

```
アイテム-トランザクション対応表M とトランザクション索引を構築
// フェーズ1：単一アイテム
for 各アイテム i ∈ I do
    TWU(i)をグローバルに計算
    if TWU(i) < θ_u then
        アイテム i を枝刈り {定理 1}
        continue
    end if
    スライディングウィンドウにより頻度密集区間 F_i を計算
    if F_i = ∅ then
        continue
    end if
    累積和 P_u({i},·) および P_TWU({i},·) を構築
    F_i 内の効用密集部分区間を探索
    if 効用密集区間が見つかった場合 then
        {i} を結果および L_1 に追加
    end if
end for
// フェーズ2以降：複数アイテム候補
for k = 2 to k_max do
    C_k ← AprioriGen(L_{k-1})
    C_k ← Prune(C_k, L_{k-1}) {Apriori性質}
    L_k ← ∅
    for 各候補 X ∈ C_k do
        単一アイテム区間から許容範囲を計算
        if 許容範囲が空 then
            continue
        end if
        P_TWU(X,·) を構築
        if max_l(WTWU(X,l)) < θ_u then
            continue {TWU枝刈り}
        end if
        共起タイムスタンプを計算
        許容範囲内で頻度密集区間を探索
        P_u(X,·) を構築
        効用密集部分区間を探索
        if 効用密集区間が見つかった場合 then
            X を結果および L_k に追加
        end if
    end for
    if L_k = ∅ then
        break
    end if
end for
```

V. 実験

UDI-Minerを以下の4つの観点から評価する：(E1) パターン復元精度、(E2) TWU枝刈り効率、(E3) スケーラビリティ、(E4) 現実的な設定における複数パターン発見。

TABLE I
E1：パターン復元結果

シード	復元	重複率	結果数	時間 (ms)
42	✓	0.970	52	39.1
123	✓	0.965	62	45.5
456	✓	0.950	74	53.1
789	✓	0.980	70	50.8
1024	✓	0.960	67	48.1
復元率: 100%		平均重複率: 0.965		

TABLE II
E2：TWU枝刈り効率

n	$\|\mathcal{I}\|$	θ_u	評価候補数		枝刈り数	高速化率
			TWU有	TWU無		
500	10	50	87	87	0	0.82×
1000	15	80	119	119	1	0.73×
2000	20	100	128	135	7	0.75×

A. 実験設定

すべての実験はPython 3.13のシングルスレッドプロセスで実施した。合成データセットは正解評価を可能にするよう制御されたパラメータで生成した。

a) データ生成。: n個のトランザクション、$|\mathcal{I}|$個のアイテムからなる効用トランザクションデータベースを生成する。各トランザクションは1–8個のアイテムを含み、ランダムな数量は$[1,5]$、外部効用は$[1,10]$から一様に抽出される。パターン注入では、対象アイテムセットを指定区間$[s^*,e^*]$内のトランザクションに確率fで挿入し、パターンアイテムの数量を係数hで増幅する。

b) デフォルトパラメータ。: 特に断りのない限り：$W = 30$–50、$\theta_f = 5$–10、$\theta_u = 50$–200、$k_{\max} = 3$–4。

B. E1：パターン復元

アイテムセット$\{2,5\}$を区間$[200,400]$に頻度0.85、数量増幅$h = 3$で注入する。$W = 40$、$\theta_f = 8$、$\theta_u = 80$、$k_{\max} = 3$でUDI-Minerを5つのランダムシードにわたり実行する。

表 Iに示すように、UDI-Minerは5回すべての実行で注入パターンを復元し、平均区間重複率は0.965であった。高い重複率は正確な時間的局所化を示している。

C. E2：TWU枝刈り効率

異なる規模でTWU枝刈りの有無による比較を行う。

表 IIに示すように、TWU枝刈りはアイテム空間の拡大に伴い効果的になる。$|\mathcal{I}| = 20$では、TWU上界により7個の候補が除去された。現時点では枝刈りの高速化効果はTWU検査のための累積和構築オーバーヘッドにより相殺されているが、より大規模な$|\mathcal{I}|$およびより厳しいθ_uの設定では、枝刈りの利点が支配的になると期待される。

D. E3：スケーラビリティ

$|\mathcal{I}| = 15$、$W = 30$、$\theta_f = 5$、$\theta_u = 50$を固定し、nを500から10,000まで変化させる。

TABLE III
E3：スケーラビリティ結果

n	時間 (ms)	結果数	評価数
500	32.6	84	87
1,000	114.7	115	119
2,000	285.6	126	128
5,000	734.8	133	134
10,000	1,501.4	140	141

表 IIIはほぼ線形のスケーラビリティを確認する。nを5,000から10,000に倍増させた場合、実行時間の増加係数は約2.04であり、$O(K \cdot n)$の計算量解析と整合する。結果数は安定化しており、固定されたデータ生成プロセスを反映している。

E. E4：複数パターン発見

半合成データセット（$n = 2{,}000$、$|\mathcal{I}| = 20$）に2つのパターンを注入する：

- パターン1：$\{3,7\}$、区間$[300,600]$、頻度0.85、数量増幅$h = 3$。
- パターン2：$\{1,5,12\}$、区間$[1200,1500]$、頻度0.80、数量増幅$h = 4$。

UDI-Miner（$W = 50$、$\theta_f = 10$、$\theta_u = 200$、$k_{\max} = 4$）は両パターンとその上位集合を正常に復元した。パターン1 $\{3,7\}$は1つの効用密集区間（最大効用1,817.0）で検出され、パターン2 $\{1,5,12\}$は1つの区間（最大効用4,190.2）で検出された。アルゴリズムはまた、$\{3,7,15\}$（12区間）や$\{1,5,12,17\}$（8区間）などの関連パターンも発見しており、より高次の効用密集共起を発見する能力を示している。

F. 考察

a) 二重条件フィルタリング。: UDIフレームワークの主要な強みは二重条件フィルタリングにある。頻度のみでは多くの低効用アイテムセットが浮上し、効用のみでは散発的な高価値トランザクションが含まれてしまう。スライディングウィンドウ内で両条件を同時に要求することで、UDIは時間的に集中した経済的に重要なパターンを特定する。

b) 累積和の効率性。: 累積和最適化により、各ウィンドウの効用計算が$O(W)$から$O(1)$に削減され、効用走査コストがウィンドウサイズに依存しなくなる。これは、実用的なアプリケーションで一般的な大きなW値（例：日次トランザクションに対する月次ウィンドウ）において極めて重要である。

c) 制約事項。: 現在のTWU枝刈りは中規模実験では限定的な効果を示す。これはランダムデータが多くのアイテムにわたって高いTWU値を蓄積する傾向があるためである。効用分布に偏りのある実際の小売データでは、より強力な枝刈り効果が期待される。

VI. 結論

本研究では、スライディングウィンドウ内で頻度と効用を同時に考慮し、高効用アイテムセットの時間的集中を検出する効用密集区間（UDI）フレームワークを提

案した。UDI問題を形式化し、ウィンドウTWUが効果的な枝刈りのための反単調上界を与えることを証明し、累積和最適化による効率的なウィンドウクエリを備えたUDI-Minerアルゴリズムを提示した。

合成データおよび半合成データに対する実験により、高い時間的精度での100%のパターン復元、アイテム空間の拡大に伴う効果的なTWUベース枝刈り、およびトランザクション数に対するほぼ線形のスケーラビリティが実証された。本フレームワークは時系列密集区間マイニングと効用考慮型分析の間の空白を橋渡しするものである。

a) 今後の課題。: 以下の方向性が検討に値する：(1) EFIMなどの一段階アルゴリズムとの統合による候補生成の回避、(2) 外部効用情報を持つ実際の小売データセットへの適用、(3) オンライン効用密集区間検出のためのストリーミング拡張、(4) 効用密集区間を説明するための外部シグナルとしての割引・販促イベントの組み込み。

REFERENCES

[1] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in *Proceedings of the 20th International Conference on Very Large Data Bases*, 1994, pp. 487–499.

[2] H. Yao, H. J. Hamilton, and C. J. Butz, "Mining itemset utilities from transaction databases," *Data & Knowledge Engineering*, vol. 59, no. 3, pp. 603–626, 2006.

[3] P. Fournier-Viger, J. C.-W. Lin, B. Vo, T. T. Chi, J. Zhang, and H. B. Le, "A survey of high utility itemset mining," *Applied Soft Computing*, vol. 83, p. 105587, 2019.

[4] M. Liu and J. Qu, "Mining high utility itemsets without candidate generation," in *Proceedings of the 21st ACM International Conference on Information and Knowledge Management*, 2012, pp. 55–64.

[5] S. Zida, P. Fournier-Viger, J. C.-W. Lin, C.-W. Wu, and V. S. Tseng, "EFIM: A highly efficient algorithm for high-utility itemset mining," in *Mexican International Conference on Artificial Intelligence*. Springer, 2015, pp. 530–546.

[6] V. S. Tseng, B.-E. Shie, C.-W. Wu, and P. S. Yu, "Efficient algorithms for mining the concise and lossless representation of high utility itemsets," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 3, pp. 726–739, 2013.

[7] G.-C. Lan, T.-P. Hong, and V. S. Tseng, "Discovery of high-utility itemsets from on-shelf time periods of products," in *Expert Systems with Applications*, vol. 41, no. 7, 2014, pp. 3403–3410.

[8] T.-L. Dam, K. Li, P. Fournier-Viger, and Q.-H. Duong, "Towards efficient mining of periodic high-utility sequential patterns," *Knowledge-Based Systems*, vol. 180, pp. 82–96, 2019.

[9] C. F. Ahmed, S. K. Tanbeer, B.-S. Jeong, and Y.-K. Lee, "Efficient tree structures for high utility pattern mining in incremental databases," in *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 12, 2009, pp. 1708–1721.